

個人履歷 + 作品集網站

React · TypeScript · Vite · TailwindCSS · GitHub Actions CI/CD

專案	brizxz.github.io — 編輯風格的個人作品集網站
類型	純靜態單頁應用(SPA),內建自動化測試與部署管線
線上	https://brizxz.github.io/
原始碼	https://github.com/brizxz/brizxz.github.io
版本	v2.0(編輯/排版風格改版)

01 專案概述

一個對外公開、隨時可分享的個人品牌網站,同時作為練習「現代軟體工程流程」的載體。網站本身是純靜態 SPA,但圍繞它建立了一條完整的 CI/CD 管線——程式碼格式、靜態檢查、型別檢查、自動化測試與自動部署,每次推送到 `main` 都會自動驗證並在一兩分鐘內上線。

核心目標

- 以資料驅動方式呈現個人簡介、技能、作品集與履歷。
- 深色 / 淺色主題切換,並做到手機 / 平板 / 桌機的 RWD 響應式排版。
- 把工程品質「自動化」——讓 CI 當守門員,沒過檢查就不准合併。
- 部署零維運:合併即上線,不需手動處理伺服器或 token。

圖 1 — 整體工程流程 · 從開發到上線



02

技術選型

類別	選擇	理由
建置工具	Vite 5	啟動快、靜態輸出最適合 GitHub Pages。
框架 / 語言	React 18 + TypeScript	生態成熟;型別讓 CI 多一道有意義的關卡。
樣式	TailwindCSS 3	暖色「紙感」中性色 + 單一強調色,克制的進場動畫。
字體	Fraunces • Inter • JetBrains Mono	襯線標題 + 無襯線內文 + 等寬標籤,營造編輯排版質感。
路由	React Router(HashRouter)	避開 GitHub Pages 對 SPA deep link 的 404 問題。
品質	ESLint + Prettier	靜態檢查 + 統一格式,避免無意義的 diff。
測試	Vitest + React Testing Library	與 Vite 原生整合;測元件行為而非實作細節。
CI/CD	GitHub Actions + deploy-pages	與 Pages 原生整合,官方 action 免自己處理 token。

03 專案結構

程式碼以「資料 / 畫面 / 邏輯」分層。所有會變動的內容集中在 `src/data/` ,改內容不需要動畫面邏輯,也方便寫測試。

圖 2 - 前端分層架構 · 由上而下的相依方向

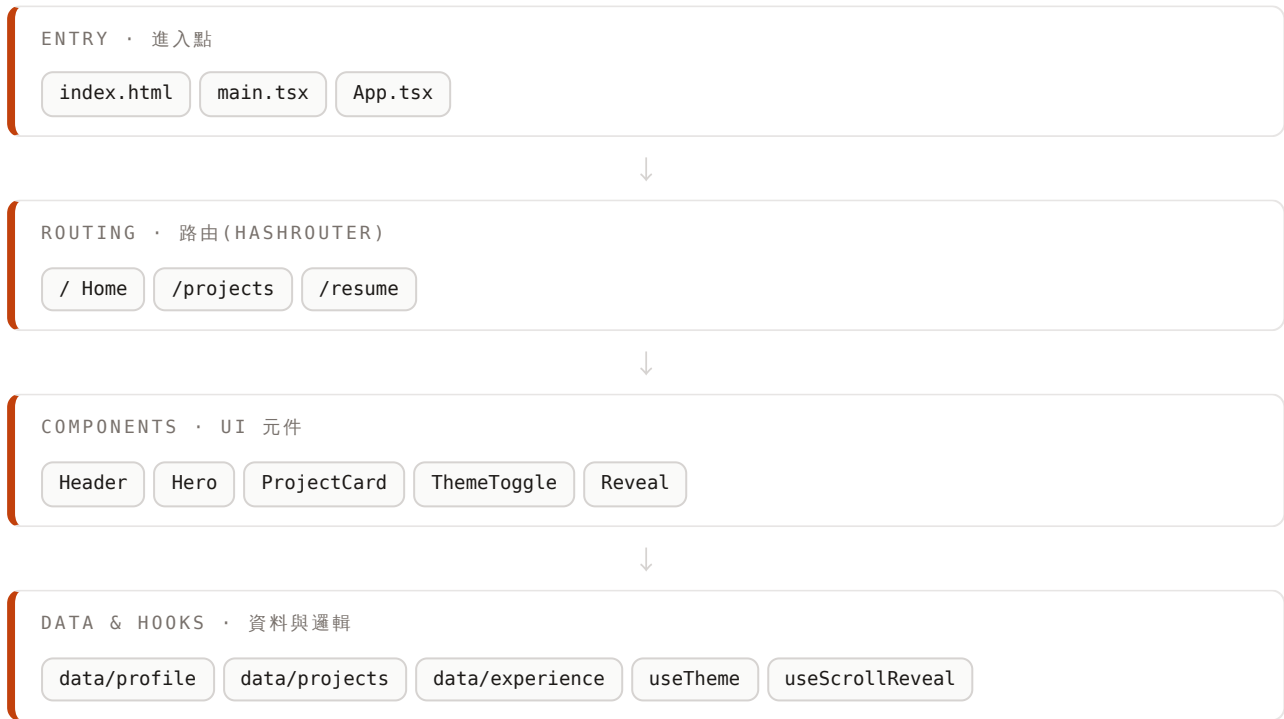


圖 3 - 目錄結構

```
github_page/
├─ .github/workflows/
│  ├─ ci.yml          # PR:format-lint→type-test→build
│  └─ deploy.yml      # push main:build→部署 Pages
├─ public/            # resume.pdf · favicon.svg
├─ src/
│  ├─ components/     # Header, Hero, ProjectCard...
│  │  └─ __tests__/   # 元件測試(Vitest)
│  ├─ pages/          # Home · Projects · Resume
│  ├─ data/            # profile / projects / experience
│  ├─ hooks/           # useTheme · useScrollReveal
│  ├─ App.tsx          # HashRouter + 路由骨架
│  └─ index.css        # Tailwind + 編輯風樣式
├─ vite.config.ts      # base:"/"(部署到 user page)
├─ tailwind.config.js  # 字體 / 色票 / darkMode:class
└─ package.json
```

■ 資料夾 · 灰字 為說明註解

04 設計系統

視覺走**編輯 / 排版(editorial)**路線:暖色紙感、強烈的字體層級、大量留白與細線分隔,刻意避開常見的漸層、光暈與 emoji 卡片,讓畫面更像「被設計過」。

面向	做法
色彩	暖灰「紙感」中性色(stone)為主 + 單一鏽橘強調色 <code>#c2410c</code> ,只用於連結、編號與重點。
字體	Fraunces 襯線標題(含斜體強調)、Inter 內文、JetBrains Mono 標籤與 metadata。
版面	編號式區塊(01-04)、12 欄格線、細線(hairline)分隔。
作品呈現	以「排版索引」列出:編號 + 語言 + 標題 + 技術標籤,取代圖片卡片。
深色模式	近黑底 + 米白字,強調色微調亮,記憶於 localStorage。
動效	僅保留克制的淡入上滑(IntersectionObserver),並尊重 <code>prefers-reduced-motion</code> 。

05 網站功能

區塊	說明
Hero 首頁	大型襯線定位句(含斜體強調)、身分 / 所在地標籤、純文字連結列。
關於 / 技能	12 欄格線:左側簡介、右側等寬技能清單。
經歷	編號式時間軸,分 Education / Project,等寬年份對齊。
作品集	排版索引列表(編號 / 語言 / 標題 / 技術 / GitHub 連結)。
履歷	編輯式兩欄排版(簡介 / 學歷 / 專案 / 技能)+ PDF 下載。
主題切換	深 / 淺色,記憶於 localStorage,首次跟隨系統偏好。

06 CI/CD 管線

兩條工作流各司其職: `ci.yml` 在 PR 階段只驗證、不部署; `deploy.yml` 只有合併進 `main` 才部署,避免半成品上線。

圖 4 - CI 管線 · 觸發於 PULL REQUEST → MAIN (快速失敗)



圖 5 - CD 管線 · 觸發於 PUSH → MAIN (或手動 WORKFLOW_DISPATCH)



關卡	工具	通過標準
格式	Prettier	無格式差異
程式碼品質	ESLint	0 error
型別	tsc	0 type error
單元 / 元件	Vitest	全綠(目前 9 個測試)
建置	Vite	成功產出 dist
部署	deploy-pages	網站可正常存取

07 GitHub Pages 部署原理

base 路徑

本站部署到使用者主頁 repo `brizxz.github.io`, 網址在根目錄, 故 `vite.config.ts` 的 `base` 設為 `"/"`。若改部署到一般 repo(`user.github.io/repo/`), 須改成 `"/repo/"`, 否則 JS / CSS 會 404、整頁空白。

實戰踩雷:Pages 的 Source。改版初期網站一片空白,原因是 Pages 的 *Source* 仍是「Deploy from a branch」,直接上架了 repo 根目錄的原始 `index.html` (指向 `/src/main.tsx` 原始碼,瀏覽器無法執行)。解法:**Settings → Pages → Source** 改成 **GitHub Actions**,並重新觸發部署,改由 `deploy.yml` 上架建置後的成品。

08 本機開發

```
npm install      # 安裝依賴
npm run dev      # 開發伺服器 http://localhost:5173
npm run build    # 型別檢查 + 建置到 dist/
npm run test     # 跑單元 / 元件測試
npm run lint     # ESLint
npm run format   # Prettier 自動格式化
```

WSL 環境提醒: 在 Windows 掛載點 `/mnt/c` 下安裝 `node_modules` 時, npm 解壓大型原生 binary (esbuild、rollup 等) 會被截斷損毀。解法: 把專案放在 WSL 原生路徑 (本專案位於 `~/projects/github_page`), 安裝與建置即正常。推送採 SSH 金鑰, 免帳密。

09 後續可擴充

- 補上待確認的個人資訊 (真實姓名、學歷年份、LinkedIn) 與真正的 PDF 履歷。
- 加入 Playwright E2E 測試與 Lighthouse CI (效能 / 無障礙自動關卡)。
- 補 Open Graph 預覽圖, 讓社群分享有縮圖。
- 在 GitHub 設定 `main` 分支保護, 要求 CI 通過才能合併。

React 18

TypeScript

Vite

TailwindCSS

React Router

Vitest

ESLint

Prettier

GitHub Actions

GitHub Pages